

Swatchbook_IR: A Clothing Information Retrieval Engine

CSD358 · Information Retrieval · Assignment 1

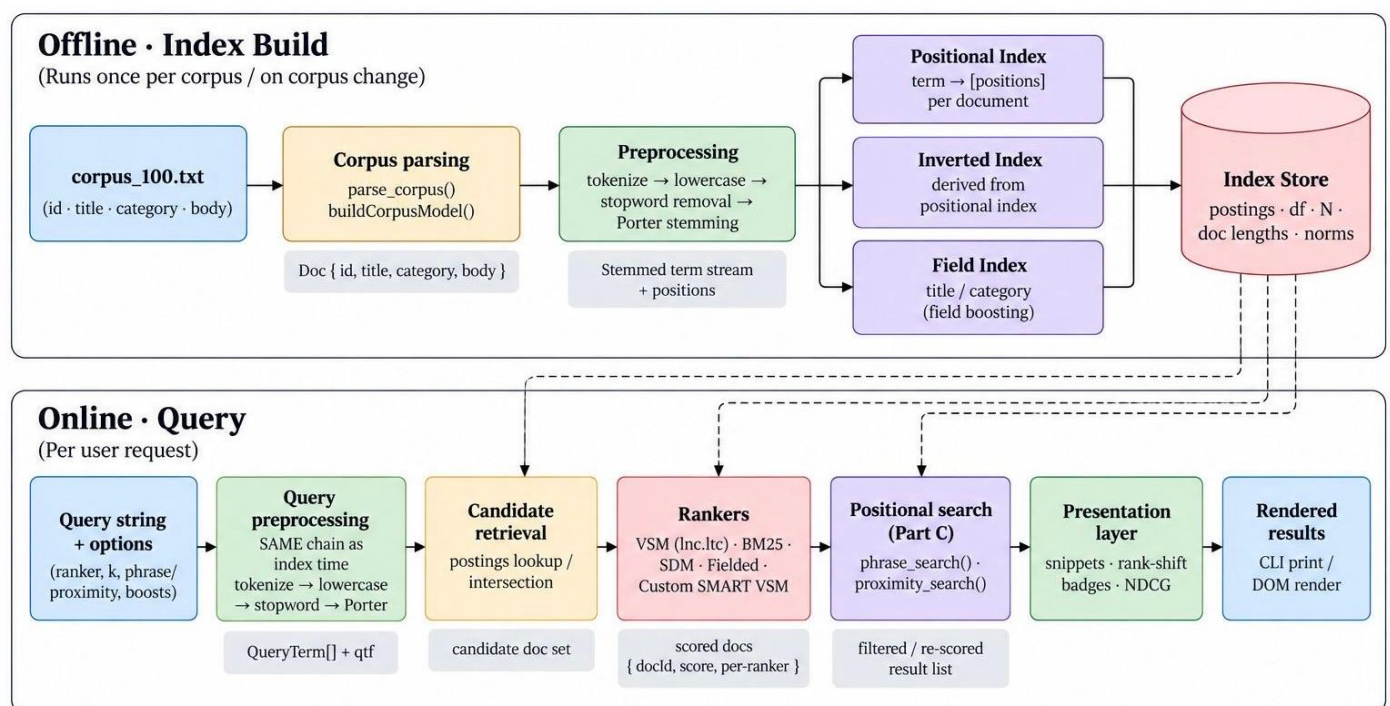
Gaurav Rai (2410110133)

Shaurya Sharma (2410110317)

GitHub repository link: https://github.com/gauravrai1704/Swatchbook_IR

1. System Architecture

The system separates cleanly into an offline indexing stage, run once per corpus (or whenever the corpus changes), and an online query stage, run once per user request. Both stages share the same preprocessing implementation, which is the single invariant that keeps term matching, phrase matching, and positional search consistent with one another.



Design invariant: Everything downstream of preprocessing depends on that step behaving identically for documents and for queries. The same `preprocess()` function is called on both corpus text and query text, never a separate function for either, which is what makes term matching, phrase matching, and position-based proximity all work correctly together.

Section 8 covers the actual workflow or the actual call sequence for each search mode.

2. Repository Structure

```
ir_clothing_search/  
├─ corpus_100.txt          # the 100-document clothing corpus
```

```

├─ swatchbook.html      # web interface (self-contained, no build step)
├─ cli.py               # terminal interface
├─ dump_indexes.py      # writes the required index deliverables
├─ src/
│   ├─ preprocess.py    # tokenize / stopword / stem pipeline
│   ├─ porter_stemmer.py # from-scratch Porter stemmer
│   ├─ index_builder.py  # inverted index + positional index construction
│   ├─ vsm.py           # Part B: lnc.ltc Vector Space Model
│   ├─ positional_search.py # Part C: phrase + proximity search
│   ├─ bm25.py          # novelty: BM25 ranking
│   ├─ sdm.py           # novelty: Sequential Dependence Model
│   ├─ fielded.py       # novelty: title/category boosted ranking
│   ├─ fuzzy.py         # novelty: typo-tolerant correction
│   ├─ snippets.py      # novelty: highlighted snippet generation
│   ├─ rank_shift.py    # novelty: rank-shift badges vs. plain VSM
│   ├─ ndcg.py          # novelty: NDCG@k computation
│   ├─ smart_vsm.py     # novelty: generalized SMART-notation VSM
│   └─ search_engine.py # facade tying everything together
├─ tests/
│   └─ run_tests.py     # Part E test harness
└─ output/
    ├─ inverted_index.txt # dictionary/inverted-index deliverable
    ├─ positional_index.txt # positional-index deliverable
    └─ test_results.md    # Part E test report

```

3. Preprocessing Pipeline (Part A)

We built the preprocessing pipeline in Python (`src/preprocess.py`) and JavaScript (the second `<script>` block in `swatchbook.html`). Both versions follow the exact same four steps in order.

1. **Tokenization:** We use a regular expression (`[a-zA-Z0-9]+`) to pull out alphanumeric words. This naturally removes punctuation and splits the text at spaces.
2. **Lowercasing:** We convert every token to lowercase before doing anything else to it.
3. **Stop-word removal:** Instead of using a standard list of 100+ stop words, we created a custom list. Since our text is generated from templates like "Made from X, this Y is designed for everyday Indian wear," our list targets the repetitive filler words that appear in almost all 100 documents (words like "the", "is", "for", "well", "made", "designed"). We intentionally kept common content words like "cotton", "wear", and "fit" because shoppers might actually search for them.
4. **Stemming:** We built the Porter (1980) algorithm from scratch (`porter_stemmer.py` and `porterStem()`). It uses five sequential suffix-stripping steps based on simple rules and consonant-vowel counts. We don't use any third-party stemming libraries.

Span-aware variant: A second function (`preprocess_with_spans()` or `preprocessWithSpans()`) runs these exact same steps but also remembers the original text and starting position of each surviving word. The position index in this output perfectly matches the standard `preprocess()` output. This allows us to map a matched stem or search position directly back to the original text for highlighting without needing extra tracking logic.

4. Index Construction (Parts A & C)

We implemented the index builder in Python (`src/index_builder.py`) and JavaScript (`buildCorpusModel()` and `buildPositionalIndex()`).

- Every document's **title + ". " + description** is preprocessed once into a token list, `doc_tokens[docid]`.
- **Positional index** is built first: `term → {df, postings: {docid: {tf, positions: [...]}}}`, where positions are 0-indexed positions in the *post-preprocessing* token stream, not raw word positions, so stopword removal shifts positions the same way for documents and queries alike.
- **Plain inverted index** (`term → {df, postings: {docid: tf}}`) required by Part A/B is then *derived* from the positional index by dropping position lists. This guarantees the two indexes can never drift out of sync, since there is a single source of truth.
- **Title/category field index** (`term → {docid: title_tf}`) is built separately, recording how often each term occurs specifically in a product's title/category, used only by the field-boosted ranker (Section 6.3).

5. Part B — Vector Space Model (Inc.Itc)

You can find this logic in Python (`src/vsm.py`) and JavaScript (the `VSMIndex` class).

Document weight ("Inc" i.e. log-tf, no idf, cosine-normalized)

$$w(d,t) = 1 + \log_{10}(tf_{d,t}) \quad \text{for } tf > 0, \text{ else } 0$$

No idf term is applied to documents. Every document's weight vector is then normalized to unit length (divided by its Euclidean norm), so the document side is purely about term saturation.

Query weight ("Itc" i.e. log-tf, idf, cosine)

$$w(q,t) = (1 + \log_{10}(tf_{q,t})) \times \log_{10}(N / df_t) \quad [N = 100]$$

This is also cosine-normalized.

Scoring: Since both vectors are already unit-normalized, we can find the cosine similarity simply by calculating the dot product of the query vector and a document vector. We don't need an extra normalization step during the search. We only score documents that share at least one term with the query by walking through the inverted index postings, which is much faster than multiplying dense matrices. We sort the results from highest score to lowest, break ties using the document ID, and return the top 10.

6. Part C — Positional Index & Phrase/Proximity Search

Implemented in `src/positional_search.py` / `phraseSearch()` and `proximitySearch()` in JavaScript.

Candidate filtering. For any multi-term positional query, the first step is always `candidate_docs()`: intersect the sets of document IDs that contain every query term at all (necessary but not sufficient), which keeps the subsequent position-by-position check cheap.

Exact phrase search (`phrase_search()`). For a phrase of n stemmed terms, for every candidate document: take every position where the first term occurs, and check whether the second term occurs at `start + 1`, the third at `start + 2`, and so on. A document matches only if this holds for the entire sequence — the distinction between an actual phrase match and “both words happen to be in this document somewhere” that Part C exists to demonstrate over Part B.

Proximity search (`proximity_search()`, the `WITHIN/k` queries). For two terms and a window k : for every pair of positions (p_a, p_b) in a candidate document, check $0 < p_b - p_a \leq k$ (ordered) or $0 < |p_b - p_a| \leq k$ (unordered). Any document with at least one qualifying pair is a match, and all qualifying pairs are kept so the app can show exactly where the match occurred.

7. Novelty Features

Eight complementary features are layered on top of the required core, each drawn from a named IR technique and disclosed where it simplifies the textbook version.

7.1 BM25 — `src/bm25.py`

Okapi BM25 (Robertson & Walker, 1994; Robertson & Zaragoza, 2009):

```
score(d,t) = idf(t) · tfd,t · (k1+1) / (tfd,t + k1 · (1 - b + b · |d|/avgdl))  
idf(t)    = log( (N - dft + 0.5) / (dft + 0.5) + 1 )
```

with $k1 = 1.5$, $b = 0.75$. Unlike `Inc.Itc`, BM25 explicitly corrects for document length via `avgdl` and applies `idf` directly rather than only on the query side — run alongside VSM as a second term-weighting philosophy on the identical index.

7.2 Sequential Dependence Model — `src/sdm.py`

Metzler & Croft (SIGIR 2005). Treats a query as evidence from three feature classes, linearly combined:

```
SDM(Q,D) = λT · Σ unigram_score + λO · Σ ordered_bigram_hits + λU · Σ unordered_window_hits
```

using Lemur/Indri’s published default weights ($\lambda_T=0.8$, $\lambda_O=0.15$, $\lambda_U=0.05$). The unigram term reuses whichever base ranker is plugged in (VSM or BM25); the ordered/unordered bigram terms come directly from the positional index — this is what lets positional evidence influence the ranking score itself, not just gate a Boolean phrase filter, and is the mechanism behind the “zero-idf” case study in Section 9.

7.3 Title/Category Field Boost — `src/fielded.py`

BM25F-inspired (Robertson, Zaragoza & Taylor, CIKM 2004). Takes any base ranker’s top results and multiplies a document’s score by $1.5\times$ for every distinct query term that also appears in that document’s title/category field. Deliberately simple — a multiplicative boost rather than a full re-derivation of BM25F’s term-frequency mixing — so it stays easy to audit.

7.4 Typo-Tolerant Fallback — `src/fuzzy.py`

Classic Levenshtein edit-distance correction. When a query term is out-of-vocabulary, every dictionary term within edit distance ≤ 2 (and within a cheap length pre-filter) is considered; the closest match, ties broken alphabetically for determinism, replaces the OOV term and the query is re-run.

7.5 Result Snippets — `src/snippets.py`, `buildSnippet()` in JS

A “best window” snippet generator, the same idea Google uses:

1. Slide a fixed-size window (16 tokens) across the document's span-annotated token stream.
2. Score each window by how many of its tokens' stems are in the query's stemmed term set.
3. Keep the highest-scoring window, expand its character range by a few characters of padding, and render the original text in that range, wrapping matched tokens.
4. Prefix/suffix an ellipsis if the snippet doesn't start/end at the document boundary.

7.6 Rank-Shift Badges — `src/rank_shift.py`

For any phrase/proximity result set, `compute_baseline_rank_map()` separately runs the same query text through plain VSM (`top_k=100`) and records each document's rank there. Each positional result is then compared: `delta = baseline_rank - positional_rank`. A positive delta means the document moved up under positional retrieval (badge ▲N); negative is ▼N; zero is =; a document VSM never ranked at all (common when every term has `idf = 0`) gets NEW. This turns the written “explain where positional information changes the result order/set” requirement into something visible on every result row.

7.7 NDCG Evaluation — `src/ndcg.py`

Järvelin & Kekäläinen (ACM TOIS, 2002), graded relevance:

$$\text{DCG}@k = \sum_{i=1..k} (2^{\text{rel}_i} - 1) / \log_2(i + 1) \quad \text{NDCG}@k = \text{DCG}@k / \text{IDCG}@k$$

where `IDCG@k` is the DCG of the same relevance grades in their ideal (best-first) order; `NDCG@k = 0` if `IDCG@k = 0` (no relevant documents graded). This module never invents relevance grades — see Section 9 for exactly where judgments come from.

7.8 Custom SMART-Notation VSM — `src/smart_vsm.py`

Generalizes `Inc.Itc` into the full SMART notation (Salton & Buckley, 1988). Any 3-character scheme code picks:

Position	Component	Codes
1	Term frequency	n natural · l log · a augmented · b boolean · L log-average
2	Document frequency	n none · t idf · p prob-idf
3	Normalization	n none · c cosine · u pivoted-unique* · b byte-size

Document and query sides are configured independently (e.g. document `Inc`, query `Itc` reconstructs the required scheme exactly — verified in `run_tests.py` to produce identical scores to `src/vsm.py`).

Honesty flag: the `u` (pivoted-unique) normalization is simplified to `1 / (unique term count)`, not the calibrated pivoted-normalization formula from Singhal, Buckley & Mitra (SIGIR 1996), which tunes a slope against the corpus-average unique-term count. This is disclosed in the `_normalize()` docstring and in the UI itself, not left implicit.

8. Chronology — What Happens When You Press Search

This is the actual call sequence for each search mode, not a simplified summary.

8.1 Free-text ranked search (Part B + novelties)

1. Form submit → `onSearchSubmit(e)` (JS) / `free_text_menu()` (CLI) reads the query string.
2. Dispatch to the active ranker chip (`vsm` / `bm25` / `sdm` / `fielded` / `custom`) selects `engine.searchVsm()`, `searchBm25()`, `searchSdm()`, `searchFielded()`, or `searchCustom()`. The CLI runs all five so they can be compared side by side.
3. Inside the ranker, the query string is preprocessed into the same term space as the index; out-of-vocabulary terms are identified by checking which stemmed query terms have no dictionary entry.
4. Query vector construction (VSM/Custom) applies the scheme's `tf/df/norm` functions; BM25 computes per-term `idf` directly; SDM gets a candidate pool from its base ranker then adds ordered/unordered bigram evidence; the Fielded ranker calls the base VSM ranker then multiplies in the title-boost factor.
5. Postings walk: for every non-zero query term, the ranker walks that term's postings list and accumulates a running per-document score — only documents sharing a term with the query are ever touched.
6. Sort & truncate: scores sorted descending, ties broken by ascending document ID, truncated to top 10.
7. Formatting: each result is enriched with title, category, rounded score, and a **snippet** via `build_snippet()/buildSnippet()`.
8. Out-of-vocabulary handling: if a query term is OOV and the result set is empty, `search_with_typo_tolerance()` runs automatically — every OOV term is Levenshtein-corrected and the corrected query is re-run.
9. Render: the web UI builds one result row per hit (rank, swatch color, title, snippet, score bar); an “inspect score” toggle lazily builds the `tf/df/idf` breakdown table only when clicked. The CLI prints the same information as plain text.
10. Compare panel (if open): `renderComparePanel()` independently re-runs the same query through every ranker at `top_k=5` and lays the columns side by side.

8.2 Phrase / proximity search (Part C + novelties)

1. Form submit in phrase mode → `runPhrase(phrase)`; in proximity mode → `runProximity(a, b, k)`.
2. Positional lookup: `search_phrase()` / `search_proximity()` preprocess the terms, call `candidate_docs()` to intersect postings, then walk position lists as in Section 6.
3. Baseline computation: the exact same query text is independently run through plain VSM at `top_k=100` via `compute_baseline_rank_map()` — a completely separate search, so the comparison stays honest.
4. Rank-shift badge assignment: each positional result's rank is compared against its rank (if any) in the VSM baseline, producing the badge described in Section 7.6.
5. Snippet generation: the same `build_snippet()` call runs here, using the phrase's (or term pair's) stemmed terms as the highlight set.
6. Render: each row gets rank, rank-shift badge, title, snippet, raw match positions, and — in the web UI only — a visual “ruler” strip showing where each hit occurred in the token stream.

8.3 Relevance & NDCG (novelty 7)

1. Switching to this tab immediately re-runs the current query, if one exists, through `runNdcg(query)`.
2. Pooling: the query is run through every ranker (VSM, BM25, SDM, Fielded, and Custom if configured) at `top_k=10` independently; the union of returned document IDs becomes the judgment pool.
3. Judging UI: one row per pooled document, each with a 4-way grade selector (0–3); an ungraded document defaults to relevance 0, the standard IR convention.

4. Live recomputation: every grade click updates the judgments map and `renderNdcgSummary()` re-runs `ndcg_at_k()` for every ranker at $k=10$ and $k=5$, with no page reload or re-fetch, since the underlying rankings are unaffected by grading.

8.4 Custom SMART VSM builder (novelty 8)

1. Chip click on “Custom SMART” reveals six dropdowns (document tf/df/norm, query tf/df/norm), defaulting to Inc (document) and Itc (query) — the required Part B scheme.
2. On any dropdown change, `applyCustomVsmScheme()` reads all six codes and calls `engine.setCustomScheme(docScheme, queryScheme)`.
3. Rebuilding the index: `SmartVSMIndex.__init__()` recomputes per-document statistics and rebuilds every document's weight vector from scratch — a full rebuild, since a different tf/df/norm choice changes every vector.
4. Re-search: if a query is active and Custom is selected, the free-text chronology in Section 8.1 runs again immediately, so results update live as dropdowns change.

9. Testing Methodology (Part E)

`tests/run_tests.py` runs 10 free-text queries through VSM (with BM25/SDM/Fielded/Custom comparisons on one), 5 exact phrase queries, 4 proximity queries at varying k , and 1 deliberately out-of-vocabulary query with its typo-tolerance correction shown — writing everything to `output/test_results.md`. Nothing here is hard-coded: every table is generated by actually calling `engine.search_*`() against the live index at report-generation time.

Case study 1 — “regular fit” (an *order* change): VSM and SDM agree on which documents match, but SDM promotes documents where the terms are adjacent, explained using SDM's own ordered-bigram hit counts.

Case study 2 — “festive wear” (a *result-set* change): every one of the 100 documents contains both words somewhere ($df = 100$ for each, so $idf = \log_{10}(100/100) = 0$), which makes Inc.Itc/BM25-style scoring mathematically unable to distinguish any document from any other — VSM's result set comes back empty. The positional index is unaffected by idf and still finds documents where the words form a genuine adjacent phrase. This is also the case that produces an all-NEW rank-shift column, since VSM never ranked any of those documents at all.

10. Where Relevance Judgments Come From

NDCG requires some notion of “relevant,” and this corpus ships with no supplied qrels file. Two different sources are used depending on context, and neither is presented as ground truth:

- **cli.py's interactive NDCG menu** asks whoever is running it to grade each pooled document 0–3 in real time — the only place judgments are genuinely a human relevance signal.
- **run_tests.py's report** needs judgments without a human in the loop, so it uses `_simulate_relevance_judgments()`: a disclosed, rule-based proxy (exact phrase match → 3, all query terms present → 2, some present → 1, none → 0). This is explicitly labeled in the generated report as a proxy for demonstrating the metric, and it is never used as a substitute for the “don't hard-code expected document IDs” rule, since it's applied uniformly at report-generation time, not a hand-picked list.

11. Known Simplifications

For a reviewer's convenience, every place this project knowingly simplifies something rather than implementing the textbook-complete version, gathered in one place (each is also flagged at its point of use in code/UI):

1. **SMART u normalization** — simplified to $1/(\text{unique-term-count})$ instead of Singhal/Buckley/Mitra's calibrated pivoted normalization.
2. **NDCG judgments in the automated test report** — a disclosed rule-based proxy, not real human relevance judgments (the CLI's interactive menu provides the real alternative).
3. **Field-boost ranker** — a multiplicative boost applied to a base ranker's score, not a full re-derivation of BM25F's term-frequency mixing across fields.
4. **SDM window sizes** (`ordered_window=2`, `unordered_window=4`) and lambda weights are Lemur/Indri's published defaults, not tuned on this specific corpus.

12. Running It

Web interface — no installation needed

```
open swatchbook.html    # or double-click it / drag into a browser
```

CLI (Python 3, standard library only)

```
python3 cli.py
```

Regenerate the index deliverables

```
python3 dump_indexes.py
```

Run the Part E test suite

```
python3 -m tests.run_tests    # writes output/test_results.md
```

13. Design Principles

- No hard-coded “correct” document IDs anywhere. Every reported result — in the tests, the CLI, or the web demo — comes from actually running the query against the live index.
- Honesty about approximations. Anywhere something is simplified (the SMART u /pivoted-unique normalization, the rule-based proxy relevance judgments), it is flagged directly in a code comment and the corresponding UI/report text, not left implicit.
- Zero third-party dependencies. Both interfaces run with nothing beyond the Python standard library and plain browser JavaScript, so a grader can run this on any machine without an install step.

14. CLI Walkthrough (Screenshots)

The screenshots below are taken from a live run of the terminal interface (`cli.py`) against the actual 100-document corpus:

```

IR_assign1 - Python cli.py - 184x51
shaurya@Shauryas-MacBook-Air-3: IR_assign1 % python3 cli.py
Loading clothing search engine (100-document corpus)...
Indexed 100 documents, 117 unique terms.

=====
1) Free-text ranked search  2) Phrase/proximity search
3) Relevance & NDCG       4) Custom SMART VSM builder  5) Quit
Choose an option: 1
Enter free-text query: cotton shirt for men

[VSM Inc.Itc] (required Part B ranking)
Rank DocID Score Category Title
1 D022 0.362 Shirt Men's Checked Cotton Shirt - White
**Men**'s Checked **Cotton** **Shirt** - White. **Men**'s Checked **Cotton** **Shirt** - White. Made from linen blend, this **shirt** is designed for everyday In...
2 D082 0.362 Shirt Men's Checked Cotton Shirt - Blue
**Men**'s Checked **Cotton** **Shirt** - Blue. **Men**'s Checked **Cotton** **Shirt** - Blue. Made from linen blend, this **shirt** is designed for everyday In...
3 D042 0.3591 Shirt Men's Checked Cotton Shirt - Maroon
**Men**'s Checked **Cotton** **Shirt** - Maroon. **Men**'s Checked **Cotton** **Shirt** - Maroon. Made from linen blend, this **shirt** is designed for everyday In...
4 D002 0.3546 Shirt Men's Checked Cotton Shirt - Black
**Men**'s Checked **Cotton** **Shirt** - Black. **Men**'s Checked **Cotton** **Shirt** - Black. Made from linen blend, this **shirt** is designed for everyday In...
5 D062 0.3546 Shirt Men's Checked Cotton Shirt - Olive
**Men**'s Checked **Cotton** **Shirt** - Olive. **Men**'s Checked **Cotton** **Shirt** - Olive. Made from linen blend, this **shirt** is designed for everyday In...
6 D001 0.354 T-Shirt Men's Cotton Crew Neck T-Shirt - Black
**Men**'s **Cotton** Crew Neck T-Shirt - Black. **Men**'s **Cotton** Crew Neck T-Shirt - Black. M...
7 D041 0.354 T-Shirt Men's Cotton Crew Neck T-Shirt - Maroon
**Men**'s **Cotton** Crew Neck T-Shirt - Maroon. **Men**'s **Cotton** Crew Neck T-Shirt - Maroon. M...
8 D061 0.354 T-Shirt Men's Cotton Crew Neck T-Shirt - White
**Men**'s **Cotton** Crew Neck T-Shirt - White. **Men**'s **Cotton** Crew Neck T-Shirt - White. M...
9 D081 0.3498 T-Shirt Men's Cotton Crew Neck T-Shirt - Mustard
**Men**'s **Cotton** Crew Neck T-Shirt - Mustard. **Men**'s **Cotton** Crew Neck T-Shirt - Mustard. M...
10 D021 0.3411 T-Shirt Men's Cotton Crew Neck T-Shirt - Sky Blue
**Men**'s **Cotton** Crew Neck T-Shirt - Sky Blue. **Men**'s **Cotton** Crew Neck T-Shirt - ...

[Novelty] BM25
Rank DocID Score Category Title
1 D022 4.9717 Shirt Men's Checked Cotton Shirt - White
**Men**'s Checked **Cotton** **Shirt** - White. **Men**'s Checked **Cotton** **Shirt** - White. Made from linen blend, this **shirt** is designed for everyday In...
2 D042 4.9717 Shirt Men's Checked Cotton Shirt - Maroon
**Men**'s Checked **Cotton** **Shirt** - Maroon. **Men**'s Checked **Cotton** **Shirt** - Maroon. Made from linen blend, this **shirt** is designed for everyday In...
3 D082 4.9717 Shirt Men's Checked Cotton Shirt - Blue
**Men**'s Checked **Cotton** **Shirt** - Blue. **Men**'s Checked **Cotton** **Shirt** - Blue. Made from linen blend, this **shirt** is designed for everyday In...
4 D001 4.9448 T-Shirt Men's Cotton Crew Neck T-Shirt - Black
**Men**'s **Cotton** Crew Neck T-Shirt - Black. **Men**'s **Cotton** Crew Neck T-Shirt - Black. M...
5 D041 4.9448 T-Shirt Men's Cotton Crew Neck T-Shirt - Maroon
**Men**'s **Cotton** Crew Neck T-Shirt - Maroon. **Men**'s **Cotton** Crew Neck T-Shirt - Maroon. M...
6 D061 4.9448 T-Shirt Men's Cotton Crew Neck T-Shirt - White
**Men**'s **Cotton** Crew Neck T-Shirt - White. **Men**'s **Cotton** Crew Neck T-Shirt - White. M...
7 D002 4.9391 Shirt Men's Checked Cotton Shirt - Black
**Men**'s Checked **Cotton** **Shirt - Black. **Men**'s Checked **Cotton** **Shirt - Black. Made from linen blend, this **shirt** is designed for everyday In...
8 D062 4.9391 Shirt Men's Checked Cotton Shirt - Olive
**Men**'s Checked **Cotton** **Shirt - Olive. **Men**'s Checked **Cotton** **Shirt - Olive. Made from linen blend, this **shirt is designed for everyday In...

```

Figure 2. Startup and a first free-text search. The CLI loads corpus_100.txt, reports that it indexed 100 documents into 117 unique terms, and then runs the query “cotton shirt for men” through the required VSM (Inc.Itc) ranker — shown here as the top-10 ranked list with score, category, and title — followed by the BM25 novelty ranker beginning to print underneath it for side-by-side comparison. This is the entry point into the free-text chronology described in Section 8.1.

```

**Men**'s **Cotton** Crew Neck T-Shirt - Black. **Men**'s **Cotton** Crew Neck T-Shirt - Black. M...
7 D041 0.354 T-Shirt Men's Cotton Crew Neck T-Shirt - Maroon
**Men**'s **Cotton** Crew Neck T-Shirt - Maroon. **Men**'s **Cotton** Crew Neck T-Shirt - Maroon. M...
8 D061 0.354 T-Shirt Men's Cotton Crew Neck T-Shirt - White
**Men**'s **Cotton** Crew Neck T-Shirt - White. **Men**'s **Cotton** Crew Neck T-Shirt - White. M...
9 D081 0.3498 T-Shirt Men's Cotton Crew Neck T-Shirt - Mustard
**Men**'s **Cotton** Crew Neck T-Shirt - Mustard. **Men**'s **Cotton** Crew Neck T-Shirt - Mustard. M...
10 D021 0.3411 T-Shirt Men's Cotton Crew Neck T-Shirt - Sky Blue
**Men**'s **Cotton** Crew Neck T-Shirt - Sky Blue. **Men**'s **Cotton** Crew Neck T-Shirt - ...

=====
1) Free-text ranked search  2) Phrase/proximity search
3) Relevance & NDCG       4) Custom SMART VSM builder  5) Quit
Choose an option: 2
Choose mode: 2
First term: cotton
Second term: shirt
Within how many positions (k): 3

[Positional] Proximity search: cotton WITHIN/3 shirt
D002 Shirt Men's Checked Cotton Shirt - Black [6 vs. plain VSM]
match positions: [(3, 4), (9, 10)]
Men's Checked **Cotton** **Shirt** - Black. Men's Checked **Cotton** **Shirt** - Black. Made from linen blend, this **shirt** is designed for everyday In...
D022 Shirt Men's Checked Cotton Shirt - White [v1 vs. plain VSM]
match positions: [(3, 4), (9, 10)]
Men's Checked **Cotton** **Shirt** - White. Men's Checked **Cotton** **Shirt** - White. Made from linen blend, this **shirt** is designed for everyday In...
D042 Shirt Men's Checked Cotton Shirt - Maroon [= vs. plain VSM]
match positions: [(3, 4), (9, 10)]
Men's Checked **Cotton** **Shirt** - Maroon. Men's Checked **Cotton** **Shirt** - Maroon. Made from linen blend, this **shirt** is designed for everyday In...
D062 Shirt Men's Checked Cotton Shirt - Olive [4 vs. plain VSM]
match positions: [(3, 4), (9, 10)]
Men's Checked **Cotton** **Shirt** - Olive. Men's Checked **Cotton** **Shirt** - Olive. Made from linen blend, this **shirt** is designed for everyday In...
D082 Shirt Men's Checked Cotton Shirt - Blue [v3 vs. plain VSM]
match positions: [(3, 4), (9, 10)]
Men's Checked **Cotton** **Shirt** - Blue. Men's Checked **Cotton** **Shirt** - Blue. Made from linen blend, this **shirt** is designed for everyday In...
D001 T-Shirt Men's Cotton Crew Neck T-Shirt - Black [v2 vs. plain VSM]
match positions: [(17, 19)]
...s **Cotton** Crew Neck T-Shirt - Black. Men's **Cotton** Crew Neck T-Shirt - Black. Made from 100% **cotton**, t...
D011 T-Shirt Men's Oversized Graphic T-Shirt - Mustard [4 vs. plain VSM]
match positions: [(14, 17)]
...s Oversized Graphic T-Shirt - Mustard. Men's Oversized Graphic T-Shirt - Mustard. Made from **cotton** blend, this t-shirt is...
D021 T-Shirt Men's Cotton Crew Neck T-Shirt - Sky Blue [42 vs. plain VSM]
match positions: [(19, 21)]
... T-Shirt - Sky Blue. Men's **Cotton** Crew Neck T-Shirt - Sky Blue. Made from 100% **cotton**, this t-shirt is...
D031 T-Shirt Men's Oversized Graphic T-Shirt - Olive Green [4 vs. plain VSM]
match positions: [(16, 19)]
...ic T-Shirt - Olive Green. Men's Oversized Graphic T-Shirt - Olive Green. Made from **cotton** blend, this t-shirt is...
D041 T-Shirt Men's Cotton Crew Neck T-Shirt - Maroon [v5 vs. plain VSM]
match positions: [(17, 19)]
...s **Cotton** Crew Neck T-Shirt - Maroon. Men's **Cotton** Crew Neck T-Shirt - Maroon. Made from 100% **cotton, t...

```

Figure 3. Switching to positional search. After the BM25 ranking for the same query finishes printing, the user picks option 2 (Phrase/proximity search) and mode 2 (Proximity search, WITHIN/k), then runs cotton WITHIN/3 shirt. Each hit is shown with its raw match positions (e.g. match positions: [(3, 4), (9, 10)]) and a rank-shift badge in brackets — [6 vs. plain VSM], [v1 vs. plain VSM], [= vs. plain VSM] — which is the live output of the baseline-comparison logic in Section 7.6.

```

IR_assign1 — Python cli.py — 184x51
match positions: [(3, 4), (9, 10)]
D042  Shirt  Men's Checked **Cotton** **Shirt** - White. Men's Checked **Cotton** **Shirt** - White. Made from linen blend, this **shirt** is designed for everyday In...
      match positions: [(3, 4), (9, 10)]
D062  Shirt  Men's Checked Cotton Shirt - Maroon [v vs. plain VSM]
      match positions: [(3, 4), (9, 10)]
D082  Shirt  Men's Checked Cotton Shirt - Olive [v3 vs. plain VSM]
      match positions: [(3, 4), (9, 10)]
D001  T-Shirt Men's Cotton Crew Neck T-Shirt - Black [v2 vs. plain VSM]
      match positions: [(17, 19)]
D011  T-Shirt Men's Oversized Graphic T-Shirt - Mustard [v4 vs. plain VSM]
      match positions: [(14, 17)]
D021  T-Shirt Men's Cotton Crew Neck T-Shirt - Sky Blue [v2 vs. plain VSM]
      match positions: [(19, 21)]
D031  T-Shirt Men's Oversized Graphic T-Shirt - Olive Green [v4 vs. plain VSM]
      match positions: [(16, 19)]
D041  T-Shirt Men's Cotton Crew Neck T-Shirt - Maroon [v5 vs. plain VSM]
      match positions: [(17, 19)]
      -s **Cotton** Crew Neck T-Shirt - Maroon. Men's Cotton Crew Neck T-Shirt - Maroon. Made from 100% cotton, t...

=====
1) Free-text ranked search  2) Phrase/proximity search
3) Relevance & NDCG       4) Custom SMART VSM builder  5) Quit
Choose an option: 2
1) Exact phrase search  2) Proximity search (WITHIN/k)
Choose mode: 1
Enter phrase (e.g. 'cotton shirt'): cotton shirt

[Positional] Exact phrase search: "cotton shirt"
D002  Shirt  Men's Checked Cotton Shirt - Black [v6 vs. plain VSM]
      match positions: [(3, 9)]
D022  Shirt  Men's Checked Cotton Shirt - Black. Men's Checked Cotton Shirt - Black. Made from linen blend, this shirt is designed for everyday In...
      match positions: [(3, 9)]
D042  Shirt  Men's Checked Cotton Shirt - Maroon [v vs. plain VSM]
      match positions: [(3, 9)]
D062  Shirt  Men's Checked Cotton Shirt - Olive [v4 vs. plain VSM]
      match positions: [(3, 9)]
D082  Shirt  Men's Checked Cotton Shirt - Blue [v3 vs. plain VSM]
      match positions: [(3, 9)]
      Men's Checked Cotton Shirt - Blue. Men's Checked Cotton Shirt - Blue. Made from linen blend, this shirt is designed for everyday In...

```

Figure 4. Remaining proximity hits, then an exact phrase search. The tail of the WITHIN/3 results is followed by switching to mode 1 (Exact phrase search) for the phrase “cotton shirt.” Every returned document again carries a rank-shift badge, and the match positions shown (e.g. [3, 9]) are exactly two apart, less far apart than the WITHIN/3 window — concretely illustrating why phrase search is a stricter condition than proximity search, as described in Section 6.

```

IR_assign1 — Python cli.py — 184x51
=====
1) Free-text ranked search  2) Phrase/proximity search
3) Relevance & NDCG       4) Custom SMART VSM builder  5) Quit
Choose an option: 4
Current scheme: Inc.Ltc
tf codes: n=natural, l=logarithm, a=augmented, b=boolean, L=log average
df codes: n=none, i=idf, p=prob idf
norm codes: n=none, c=cosine, u=uniquated unique*, b=byte size
Document scheme (3 chars, e.g. Inc) [Inc]: bnn
Query scheme (3 chars, e.g. Ltc) [Ltc]: bnn
Custom scheme set to bnn.bnn. Use option 1 (free-text search) to see it ranked.

=====
1) Free-text ranked search  2) Phrase/proximity search
3) Relevance & NDCG       4) Custom SMART VSM builder  5) Quit
Choose an option: 1
Enter free-text query: cotton shirt

[VSM Inc.Ltc] (Required Part B ranking)
Rank DocID Score Category Title
1 D022 0.295 Shirt Men's Checked Cotton Shirt - White
2 D082 0.295 Shirt Men's Checked Cotton Shirt - White. Men's Checked Cotton Shirt - White. Made from linen blend, this shirt is designed for everyday In...
3 D042 0.2926 Shirt Men's Checked Cotton Shirt - Maroon
4 D001 0.2911 T-Shirt Men's Cotton Crew Neck T-Shirt - Black
5 D041 0.2911 T-Shirt Men's Cotton Crew Neck T-Shirt - Maroon
6 D061 0.2911 T-Shirt Men's Cotton Crew Neck T-Shirt - White
7 D002 0.2889 Shirt Men's Checked Cotton Shirt - Black
8 D062 0.2889 Shirt Men's Checked Cotton Shirt - Olive
9 D081 0.2877 T-Shirt Men's Cotton Crew Neck T-Shirt - Mustard
10 D021 0.2806 T-Shirt Men's Cotton Crew Neck T-Shirt - Sky Blue
    T-Shirt - Sky Blue. Men's Cotton Crew Neck T-Shirt - Sky Blue. Made from 100% cotton, this t-shirt is...

[Novelty] BM25
Rank DocID Score Category Title
1 D001 3.4919 T-Shirt Men's Cotton Crew Neck T-Shirt - Black
2 D041 3.4919 T-Shirt Men's Cotton Crew Neck T-Shirt - Maroon
3 D061 3.4919 T-Shirt Men's Cotton Crew Neck T-Shirt - White
4 D081 3.4722 T-Shirt Men's Cotton Crew Neck T-Shirt - Mustard
5 D022 3.4642 Shirt Men's Checked Cotton Shirt - White

```

Figure 5. Custom SMART VSM builder. Choosing option 4 reveals the six *tf/df/norm* codes; both the document and query schemes are set to *bnn.bnn* (boolean term frequency, no *idf*, no normalization) instead of the default *Inc.Ltc*, and the tool confirms “Custom scheme set to *bnn.bnn*.” The next free-text search for “cotton shirt” then shows the required VSM (*Inc.Ltc*) ranking and the BM25 novelty ranking, ready to be compared against the custom scheme once it's selected — this is the live version of the rebuild-from-scratch workflow in Section 8.4.

```
IR_assign1 - Python cli.py - 184x51
...s Oversized Graphic T-Shirt - Mustard. Men's Oversized Graphic T-Shirt - Mustard. Made from cotton blend, this t-shirt is...
4 D021 2.0 T-Shirt Men's Cotton Crew Neck T-Shirt - Sky Blue
- T-shirt - Sky Blue. Men's Cotton Crew Neck T-Shirt - Sky Blue. Made from 100% cotton, this t-shirt is...
D022 2.0 Shirt Men's Checked Cotton Shirt - White
Men's Checked Cotton Shirt - White. Men's Checked Cotton Shirt - White. Made from linen blend, this shirt is designed for everyday In...
6 D031 2.0 T-Shirt Men's Oversized Graphic T-Shirt - Olive Green
-ic T-Shirt - Olive Green. Men's Oversized Graphic T-Shirt - Olive Green. Made from cotton blend, this t-shirt is...
7 D041 2.0 T-Shirt Men's Cotton Crew Neck T-Shirt - Maroon
-s Cotton Crew Neck T-Shirt - Maroon. Men's Cotton Crew Neck T-Shirt - Maroon. Made from 100% cotton, t...
8 D042 2.0 Shirt Men's Checked Cotton Shirt - Maroon
Men's Checked Cotton Shirt - Maroon. Men's Checked Cotton Shirt - Maroon. Made from linen blend, this shirt is designed for everyday In...
9 D051 2.0 T-Shirt Men's Oversized Graphic T-Shirt - Navy Blue
-ic T-Shirt - Navy Blue. Men's Oversized Graphic T-Shirt - Navy Blue. Made from cotton blend, this t-shirt is...
10 D061 2.0 T-Shirt Men's Cotton Crew Neck T-Shirt - White
-s Cotton Crew Neck T-Shirt - White. Men's Cotton Crew Neck T-Shirt - White. Made from 100% cotton, t...

=====
1) Free-text ranked search 2) Phrase/proximity search
3) Relevance & NDCG 4) Custom SMART VSM builder 5) Quit
Choose an option: 3
Enter free-text query to judge: winter jacket

Pooled top-10 documents across 5 rankers (10 unique). Grade each 0 (not relevant) to 3 (highly relevant); blank = 0.
D008 - Women's Casual Puffer Jacket - Olive
relevance (0-3): 2
D018 - Women's Quilted Winter Jacket - Beige
relevance (0-3): 3
D028 - Women's Casual Puffer Jacket - Black
relevance (0-3): 2
D038 - Women's Quilted Winter Jacket - Olive
relevance (0-3): 3
D048 - Women's Casual Puffer Jacket - Beige
relevance (0-3): 2
D058 - Women's Quilted Winter Jacket - Black
relevance (0-3): 3
D068 - Women's Casual Puffer Jacket - Olive
relevance (0-3): 2
D078 - Women's Quilted Winter Jacket - Beige
relevance (0-3): 3
D088 - Women's Casual Puffer Jacket - Black
relevance (0-3): 2
D098 - Women's Quilted Winter Jacket - Olive
relevance (0-3): 3

[NDCG] "winter jacket" -- your own relevance grades, nothing hard-coded
Ranker NDCG@10 NDCG@5
VSM (lnc.ltc) 1.0000 1.0000
BM25 1.0000 1.0000
SVM 1.0000 1.0000
Title-boosted VSM 1.0000 1.0000
Custom (bnn.bnn) 0.8543 0.6343
```

Figure 6. Relevance judging and NDCG. Choosing option 3 (Relevance & NDCG) for the query “winter jacket” pools the top-10 documents returned by all five rankers (10 unique documents here) and asks the user to grade each one 0–3 by hand — a genuine human relevance signal, as opposed to the rule-based proxy used by the automated test report (Section 10). Once every document is graded, NDCG@10 and NDCG@5 are computed live for every ranker, including the Custom (bnn.bnn) scheme configured in the previous screenshot, without re-running any of the underlying searches.

15. References

Papers and texts that the ranking algorithms and evaluation methods in this project are drawn from, in the order they first appear above:

- Porter, M. F. (1980). An algorithm for suffix stripping. *Program*, 14(3), 130–137.
- Robertson, S. E., & Walker, S. (1994). Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval. *Proceedings of SIGIR 1994*.
- Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Metzler, D., & Croft, W. B. (2005). A Markov random field model for term dependencies. *Proceedings of SIGIR 2005*.
- Robertson, S., Zaragoza, H., & Taylor, M. (2004). Simple BM25 extension to multiple weighted fields. *Proceedings of CIKM 2004*.
- Järvelin, K., & Kekäläinen, J. (2002). Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4), 422–446.
- Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5), 513–523.
- Singhal, A., Buckley, C., & Mitra, M. (1996). Pivoted document length normalization. *Proceedings of SIGIR 1996*.